

Query Query History

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE books (

id SERIAL PRIMARY KEY,

title VARCHAR(200) NOT NULL,

author VARCHAR(200),

description TEXT

);

TRUNCATE TABLE books RESTART IDENTITY;

INSERT INTO books (title, author, description) VALUES

('The Martian', 'Andy Weir', 'A stranded astronaut struggles to survive on Mars.')

('Dune', 'Frank Herbert', 'A science-fiction story involving planets, politics and survival

('Pride and Prejudice', 'Jane Austen', 'A classic story of relationships, family and societ

('The Hobbit', 'J.R.R. Tolkien', 'A fantasy adventure involving a journey, creatures and a

('Foundation', 'Isaac Asimov', 'A science-fiction story about humanity and the future of ci

SELECT * FROM books;

ALTER TABLE books ADD COLUMN embedding VECTOR(3);

Data Output Messages Notifications

NOTICE: extension "vector" already exists, skipping

ERROR: relation "books" already exists

SQL state: 42P07

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

CREATE TABLE books (

id SERIAL PRIMARY KEY,

title VARCHAR(200) NOT NULL,

author VARCHAR(200),

description TEXT

);

TRUNCATE TABLE books RESTART IDENTITY;

INSERT INTO books (title, author, description) VALUES

('The Martian', 'Andy Weir', 'A stranded astronaut struggles to survive on Mars.')

('Dune', 'Frank Herbert', 'A science-fiction story involving planets, politics and survival

('Pride and Prejudice', 'Jane Austen', 'A classic story of relationships, family and societ

('The Hobbit', 'J.R.R. Tolkien', 'A fantasy adventure involving a journey, creatures and a

('Foundation', 'Isaac Asimov', 'A science-fiction story about humanity and the future of ci

SELECT * FROM books;

ALTER TABLE books ADD COLUMN embedding VECTOR(3);

Data Output Messages Notifications

ERROR: relation "books" already exists

SQL state: 42P07

```
1 CREATE EXTENSION IF NOT EXISTS vector;
2
3 CREATE TABLE books (
4     id SERIAL PRIMARY KEY,
5     title VARCHAR(200) NOT NULL,
6     author VARCHAR(200),
7     description TEXT
8 );
9 TRUNCATE TABLE books RESTART IDENTITY;
10 INSERT INTO books (title, author, description) VALUES
11 ('The Martian', 'Andy Weir', 'A stranded astronaut struggles to survive on Mars. '),
12 ('Dune', 'Frank Herbert', 'A science-fiction story involving planets, politics and survival'),
13 ('Pride and Prejudice', 'Jane Austen', 'A classic story of relationships, family and society'),
14 ('The Hobbit', 'J.R.R. Tolkien', 'A fantasy adventure involving a journey, creatures and a dangerous quest'),
15 ('Foundation', 'Isaac Asimov', 'A science-fiction story about humanity and the future of civilization');
16
17 SELECT * FROM books;
18
19 ALTER TABLE books ADD COLUMN embedding VECTOR(3);
```

Data Output Messages Notifications

INSERT 0 5

Query returned successfully in 109 msec.

```
14 ('The Hobbit', 'J.R.R. Tolkien', 'A fantasy adventure involving a journey, creatures and a dangerous quest'),
15 ('Foundation', 'Isaac Asimov', 'A science-fiction story about humanity and the future of civilization');
16
17 SELECT * FROM books;
18
19 ALTER TABLE books ADD COLUMN embedding VECTOR(3);
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	id [PK] integer	title character varying (200)	author character varying (200)	description text	embedding vector
1	1	The Martian	Andy Weir	A stranded astronaut struggles to survive on Mars.	[null]
2	2	Dune	Frank Herbert	A science-fiction story involving planets, politics and survival.	[null]
3	3	Pride and Prejudice	Jane Austen	A classic story of relationships, family and society.	[null]
4	4	The Hobbit	J.R.R. Tolkien	A fantasy adventure involving a journey, creatures and a dangerous qu...	[null]
5	5	Foundation	Isaac Asimov	A science-fiction story about humanity and the future of civilization.	[null]

```
17 SELECT * FROM books;
18
19 ALTER TABLE books ADD COLUMN embedding VECTOR(3);
20
21 UPDATE books
22 SET embedding = CASE title
23     WHEN 'The Martian' THEN '[0.10, 0.90, 0.20]':vector
24     WHEN 'Dune' THEN '[0.20, 0.85, 0.25]':vector
25     WHEN 'Pride and Prejudice' THEN '[0.80, 0.10, 0.70]':vector
26     WHEN 'The Hobbit' THEN '[0.30, 0.20, 0.90]':vector
27     WHEN 'Foundation' THEN '[0.15, 0.80, 0.30]':vector
28 END;
```

Data Output Messages Notifications

ERROR: column "embedding" of relation "books" already exists

SQL state: 42701

```
20
21 UPDATE books
22 SET embedding = CASE title
23 WHEN 'The Martian' THEN '[0.10, 0.90, 0.20]':vector
24 WHEN 'Dune' THEN '[0.20, 0.85, 0.25]':vector
25 WHEN 'Pride and Prejudice' THEN '[0.80, 0.10, 0.70]':vector
26 WHEN 'The Hobbit' THEN '[0.30, 0.20, 0.90]':vector
27 WHEN 'Foundation' THEN '[0.15, 0.80, 0.30]':vector
28 END;
```

Data Output Messages Notifications

UPDATE 5

Query returned successfully in 194 msec.

```
SELECT id, title, embedding FROM books;

SELECT
  id,
  title,
  embedding <-> '[0.10, 0.85, 0.20]':vector AS l2_distance
FROM books
WHERE embedding IS NOT NULL
ORDER BY embedding <-> '[0.10, 0.85, 0.20]':vector
LIMIT 3;

SELECT
  id,
  title,
  embedding <=> '[0.10, 0.85, 0.20]':vector AS cosine distance
```

Output Messages Notifications

Showing rows:

id [PK] integer	title character varying (200)	embedding vector
1	The Martian	[0.1,0.9,0.2]
2	Dune	[0.2,0.85,0.2...
3	Pride and Prejudice	[0.8,0.1,0.7]
4	The Hobbit	[0.3,0.2,0.9]
5	Foundation	[0.15,0.8,0.3]

```
31
32 SELECT
33   id,
34   title,
35   embedding <-> '[0.10, 0.85, 0.20]':vector AS l2_distance
36 FROM books
37 WHERE embedding IS NOT NULL
38 ORDER BY embedding <-> '[0.10, 0.85, 0.20]':vector
39 LIMIT 3;
40
41 SELECT
42   id,
43   title,
44   embedding <=> '[0.10, 0.85, 0.20]':vector AS cosine distance
```

Data Output Messages Notifications

Showing rows: 1 to 3

	id [PK] integer	title character varying (200)	l2_distance double precision
1	1	The Martian	0.049999952875055514
2	2	Dune	0.11180339970798972
3	5	Foundation	0.12247450097883303

```
SELECT
  id,
  title,
  embedding <=> '[0.10, 0.85, 0.20]':vector AS cosine_distance
FROM books
WHERE embedding IS NOT NULL
ORDER BY embedding <=> '[0.10, 0.85, 0.20]':vector
LIMIT 3;
```

```
SELECT
  id,
  title,
  1 - (embedding <=> '[0.10, 0.85, 0.20]':vector) AS cosine_similarity
```

Data Output Messages Notifications

Showing rows: 1 to 3

id	title	cosine_distance
[PK] integer	character varying (200)	double precision
1	The Martian	9.408262185794936e-05
2	Dune	0.0072885302161147525
5	Foundation	0.009750970439461937

```
SELECT
  id,
  title,
  1 - (embedding <=> '[0.10, 0.85, 0.20]':vector) AS cosine_similarity
FROM books
WHERE embedding IS NOT NULL
ORDER BY embedding <=> '[0.10, 0.85, 0.20]':vector
LIMIT 3;
```

Data Output Messages Notifications

Showing rows: 1 to 3

id	title	cosine_similarity
[PK] integer	character varying (200)	double precision
1	The Martian	0.999905917378142
2	Dune	0.9927114697838852
5	Foundation	0.9902490295605381

```
1 openapi: 3.0.3
2 info:
3   title: BookFlow Search API
4   version: 1.0.0
5   description: API contract for semantic book search
6
7 paths:
8   /v1/books/search:
9     get:
10      summary: Search books semantically
11      operationId: searchBooks
12      parameters:
13        - name: q
14          in: query
15          required: true
16          description: Natural-language search query
17          schema:
18            type: string
19            minLength: 1
20
21        - name: limit
22          in: query
23          required: false
24          description: Maximum number of results
25          schema:
26            type: integer
27            minimum: 1
28            maximum: 20
29            default: 3
30
31      responses:
32        '200':
33          description: Search results
34          content:
35            application/json:
36              schema:
37                type: object
38                required: [query, results]
39                properties:
40                  query:
41                    type: string
42                  results:
```

BookFlow Search API 1.0.0 OAS 3.0

API contract for semantic book search

default

GET /v1/books/search Search books semantically

Parameters

Cancel

Name	Description
q required	Natural-language search query
string (query)	Stories about space
limit	Maximum number of results
integer (query)	3

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'https://editor.swagger.io/v1/books/search?q=Stories%20about%20space&limit=3' \
  -H 'accept: application/json'
```

```
31 responses:
32   '200':
33     description: Search results
34     content:
35       application/json:
36         schema:
37           type: object
38           required: [query, results]
39           properties:
40             query:
41               type: string
42             results:
43               type: array
44               items:
45                 type: object
46                 required: [id, title, score]
47                 properties:
48                   id:
49                     type: integer
50                   title:
51                     type: string
52                   score:
53                     type: number
54                     format: float
55
56   '400':
57     description: Bad Request
58     content:
59       application/json:
60         schema:
61           type: object
62           required: [error]
63           properties:
64             error:
65               type: object
66               required: [code, message]
67               properties:
68                 code:
69                   type: string
70                 message:
71                   type: string
```

Curl

```
curl -X 'GET' \
  'https://editor.swagger.io/v1/books/search?q=Stories%20about%20space&limit=3' \
  -H 'accept: application/json'
```

Request URL

https://editor.swagger.io/v1/books/search?q=Stories%20about%20space&limit=3

Server response

Code

Details

404

undocumented

Error: response status is 404

Response body

```
<html>
<head><title>404 Not Found</title></head>
<body>
<center><h1>404 Not Found</h1></center>
<br><center>nginx/1.31.5</center>
</body>
</html>
<!-- a padding to disable MSIE and Chrome friendly error page -->
<!-- a padding to disable MSIE and Chrome friendly error page -->
<!-- a padding to disable MSIE and Chrome friendly error page -->
<!-- a padding to disable MSIE and Chrome friendly error page -->
<!-- a padding to disable MSIE and Chrome friendly error page -->
<!-- a padding to disable MSIE and Chrome friendly error page -->
```

Response headers

```
content-length: 355
content-type: text/html
date: Sat, 12 Sep 2020 06:39:07 GMT
server: nginx/1.31.5
```

Responses

Code	Description	Links
200	Search results	No links

Media type

application/json

Controls Accept header

Example Value... Schema

Conclusion:

In this experiment, I successfully created the bookflow_db database using PostgreSQL and enabled the pgvector extension. I created the books table and populated it with sample book data and 3-dimensional mock embeddings. I performed semantic similarity searches using L2 distance and cosine distance and converted cosine distance into cosine similarity scores to rank the most relevant books. Finally, I designed and verified the BookFlow Search API contract using OpenAPI/Swagger, including the 200 Search Results response and 400 Bad Request response. Thus, the database, vector-search functionality, and API blueprint required for the BookFlow intelligent catalog were successfully completed.